

Quiniela

Repositorio de documentacion tecnica

12 Decisiones por Sprint 06

12_Decisiones_por_Sprint

Version: 0.1.0

Estado: Borrador operativo

Fecha: 12 de mayo de 2026

Documento interno de trabajo

Índice general

1	Sprint 6	2
1.1	Contexto	2
1.2	Resumen ejecutivo	2
1.3	Leaderboard y score visible	5
1.4	Correcciones y recalculo	6
1.5	Scoring persistido y edge cases	10
1.6	Leaderboard backend y desempates cerrados	13
1.7	Fronteras tecnicas obligatorias	17
1.8	Alineaciones obligatorias antes de ticketear Sprint 6	17
1.9	Notas	17

1 Sprint 6

1.1 Contexto

Este documento registra las decisiones cerradas necesarias para planificar y ejecutar Sprint 6 sin ambigüedad. Su objetivo es cerrar los bloqueadores de producto, UX, frontend y backend que afectan scoring, recalculation y leaderboard, sin empujar decisiones al momento de implementación.

Enfoque del sprint. Sprint 6 mantiene como goal oficial cerrar el primer ciclo competitivo completo: picks, resultados oficiales, scoring y leaderboard visible. Las decisiones registradas aquí no cambian ese goal; lo vuelven ejecutable sin improvisación.

1.2 Resumen ejecutivo

Decision	Verdicto
Superficie visible del leaderboard	Una sola superficie autenticada de ranking con filtros para general, fase y jornada
Precision visual de puntajes no enteros	Enteros sin decimales; valores fraccionarios con exactamente dos decimales visibles
Desglose visual de puntos	Total visible como dato primario; desglose compacto en ranking y desglose expandido solo en perfil o detalle si backend lo expone
Tratamiento de picks especiales en leaderboard filtrado	El ranking filtrado por fase o jornada excluye del puntaje visible los picks especiales
Feedback de resultado corregido	Badge persistente en el partido corregido y aviso breve cuando cambie puntaje o ranking
Modelo de recalculation de Sprint 6	Recalculation automatico selectivo por partido afectado; recalculation global admin queda fuera del sprint
Autoridad oficial de scoring y ranking	Backend es la unica autoridad oficial; frontend no puede calcular ni ordenar ranking por su cuenta
Politica de refresco frontend tras correccion	Invalida resultados, calendario afectado, score visible y leaderboard activo; no consume campos no garantizados por contrato
Partido cancelado con resultado persistido	Mientras el partido siga en CANCELADO, no es utilizable para scoring aunque exista un <code>Result</code> persistido

Autoridad operativa entre estado de partido y resultado persistido	Para scoring manda la combinacion consistente entre estado funcional y verdad oficial vigente utilizable; una contradiccion requiere correccion administrativa auditada
Tipo persistente de <code>ScoreEntry.points</code>	Para Sprint 6, <code>ScoreEntry.points</code> debe persistirse como <code>Float</code> ; <code>Int</code> contradice el scoring fraccional oficial sin redondeo
Contrato de <code>ScoreEntry.reason</code>	Campo obligatorio sin enum cerrado en Sprint 6; debe conservar contexto funcional suficiente del origen del puntaje
Contenido minimo de <code>ScoreEntry.reason</code>	Debe distinguir al menos ausencia valida de pick, outcome correcto, marcador exacto y cancelacion sin resultado utilizable
Politica de actualizacion de <code>ScoreEntry.reason</code>	La fila vigente de <code>ScoreEntry</code> se actualiza con el motivo final vigente; el historial queda en auditoria, no versionado dentro de la misma fila
Contrato backend de leaderboard general (QH-151)	<code>GET /leaderboard</code> autenticado, contrato fijo de fila y posicion resuelta por backend; incluye solo usuarios <code>ACTIVO</code> , incluyendo activos con 0 puntos
Orden operativo base de leaderboard en QH-151	Mientras QH-152 no este aplicado: <code>total DESC</code> , <code>user.createdAt ASC</code> , <code>user.id ASC</code> , con posicion secuencial <code>1..n</code>
Jerarquia oficial de desempates (QH-152)	Sobre empate en total: <code>EXACT_SCORE DESC</code> , <code>OUTCOME_HITS DESC</code> , <code>KO_POINTS DESC</code> , registro mas temprano y fallback tecnico <code>user.id ASC</code>
Semantica operativa de conteos de desempate en QH-152	<code>EXACT_SCORE</code> tambien cuenta como acierto de outcome; especiales suman al total general pero no participan en criterios de desempate
Modelo semantico de fase y jornada para QH-153	<code>fase</code> se implementa con <code>Phase</code> ; <code>jornada</code> es concepto distinto implementado como <code>Match.matchday</code> persistente
Contrato HTTP de leaderboard filtrado en QH-153	Se reutiliza <code>GET /leaderboard</code> con query params mutuamente excluyentes <code>phaseId</code> o <code>matchday</code> ; sin filtros devuelve vista general
Semantica de score y filtros en QH-153	En respuesta filtrada, <code>score.total</code> significa puntaje del scope consultado; especiales quedan completamente fuera del response filtrado
Desempates y presencia de usuarios en leaderboard filtrado	Los desempates se recalculan solo con entries del scope filtrado; usuarios <code>ACTIVO</code> con 0 puntos en el scope siguen apareciendo

Errores y alcance tecnico de QH-153	<code>phaseId</code> inexistente responde 404; <code>matchday</code> sin resultados responde array vacio; el ticket incluye backend, tests, OpenAPI, Postman y seed minima
Trigger de recalc selectivo en QH-150	Se dispara en manual y sync, pero solo ante cambio efectivo del marcador oficial vigente
Definicion de cambio efectivo en QH-150	Solo cambia si cambia <code>homeGoals</code> y/o <code>awayGoals</code> ; metadata sin cambio de marcador no recalcula
Alcance del recalc selectivo en QH-150	Solo recalcula <code>ScoreEntry</code> de picks del mismo <code>matchId</code> ; no hay recalc global, por torneo ni por usuario
Contrato interno minimo del recalc selectivo	Input canonico con <code>matchId</code> , origen, causa y contexto; output con contadores, afectados y ventana temporal
Trazabilidad minima de recalc selectivo	Debe registrar <code>AuditEvent</code> con accion <code>match_score_recalculated</code> y contadores del resultado
Politica transaccional del recalc selectivo	El resultado oficial se persiste primero; el recalc corre despues y su fallo no revierte la verdad oficial vigente
Error publico ante fallo de recalc selectivo	Devuelve <code>internal_error</code> ; el detalle fino vive en auditoria y logging interno
Idempotencia operativa del recalc selectivo	Un re-run sobre el mismo estado final converge sin cambios efectivos y deja trazabilidad del intento
Relacion de QH-150 con predicciones especiales	QH-150 recalcula solo picks por partido; excluye <code>specialPredictionId</code>
Evidencia minima backend para QH-150	Debe cubrir cambio efectivo manual y sync, no-op por metadata, idempotencia, aislamiento por match y auditoria

1.3 Leaderboard y score visible

Decision. Superficie visible del leaderboard

Verdicto. Sprint 6 usa una sola superficie autenticada de ranking con filtros para vista general, por fase y por jornada. No se crean pantallas top-level separadas para cada corte del leaderboard.

Por que. Desde principios de HCI, una sola superficie reduce costo de navegacion, mejora descubribilidad y evita que el usuario tenga que aprender varias pantallas para el mismo objetivo funcional. Desde software engineering, tambien reduce duplicacion de estados, queries y componentes.

Impacto. Frontend planifica una sola pagina o modulo de leaderboard con controles de filtro visibles. Backend debe soportar el mismo modelo conceptual de cortes sin obligar al cliente a recomponer ranking localmente.

Documento dueno. 04_Frontend/tex/04_02_Flujos_y_Paginas.tex y
03_Backend/tex/03_04_Endpoints_Conceptuales.tex

Decision. Precision visual de puntajes no enteros

Verdicto. El valor oficial de scoring conserva precision exacta en backend. En UI, los puntajes enteros se muestran sin decimales y los puntajes fraccionarios se muestran con exactamente dos decimales visibles.

Por que. El dominio ya habia cerrado que backend no redondea. A nivel de UX, mostrar enteros sin ruido visual mejora legibilidad; mostrar fracciones con dos decimales evita ambigüedad, mantiene consistencia visual y representa correctamente valores esperables como 6.25, 3.75 o 4.50.

Impacto. El frontend no debe truncar ni redondear de forma que altere la percepcion del valor oficial. El orden del leaderboard sigue viniendo del backend; el formato visual no redefine ranking.

Documento dueno. 01_Producto_y_Reglas/tex/01_04_Scoring_y_Desempates.tex y
04_Frontend/tex/04_03_Estado_Validacion_y_Datos.tex

Decision. Desglose visual de puntos

Verdicto. En leaderboard, el total visible del scope activo es el dato primario. El desglose se presenta como contexto secundario compacto. El desglose expandido vive solo en perfil o detalle de rendimiento si el backend expone ese contrato.

Por que. En superficies densas como un ranking, HCI recomienda priorizar escaneo rapido y comparacion clara. Un detalle excesivo dentro de cada fila degrada legibilidad. Separar total primario y desglose secundario mantiene transparencia sin sobrecargar la tabla.

Impacto. La vista general puede mostrar contexto compacto del tipo **Partidos + Especiales**. La vista de perfil o detalle puede mostrar desglose mas amplio cuando exista contrato oficial. Sprint 6 no obliga a un breakdown exhaustivo embebido en cada fila del leaderboard.

Documento dueno. 01_Producto_y_Reglas/tex/01_04_Scoring_y_Desempates.tex y
04_Frontend/tex/04_02_Flujos_y_Paginas.tex

Decision. Picks especiales en leaderboard filtrado

Verdicto. El leaderboard filtrado por fase o jornada muestra solo puntaje atribuible al scope filtrado. Los puntos de **Campeon** y **Goleador** no forman parte del puntaje visible de una vista filtrada.

Por que. Las predicciones especiales pertenecen al acumulado del torneo completo y no a una jornada o fase puntual. Incluirlas dentro de un corte filtrado rompe el modelo mental del usuario y dificulta interpretar por que una posicion cambia en esa vista.

Impacto. El leaderboard general sigue usando el total acumulado del torneo. Una vista filtrada puede mostrar el total general como contexto secundario si el producto lo considera util, pero el puntaje principal del ranking filtrado excluye picks especiales.

Documento dueno. 01_Producto_y_Reglas/tex/01_04_Scoring_y_Desempates.tex

1.4 Correcciones y recalcu

Decision. Feedback visible de resultado corregido

Verdicto. Cuando un resultado oficial fue corregido, el partido corregido debe mostrar badge persistente. Si esa correccion cambia puntaje o ranking del usuario, el frontend debe mostrar ademas un aviso breve y explicito.

Por que. Desde principios de feedback y visibilidad del estado del sistema, el usuario debe entender que hubo una correccion y si tuvo consecuencia competitiva. El badge persistente da trazabilidad visible; el aviso breve comunica impacto sin convertir el leaderboard en una consola de auditoria.

Impacto. El calendario o card del partido corregido es la superficie primaria de contexto persistente. El leaderboard refleja el nuevo dato por refresco, pero no es la superficie principal de notificacion.

Documento dueno. 04_Frontend/tex/04_02_Flujos_y_Paginas.tex y
04_Frontend/tex/04_03_Estado_Validacion_y_Datos.tex

Decision. Modelo de recalcu de Sprint 6

Verdicto. En Sprint 6, el modelo obligatorio es recalcu automatico selectivo por partido afectado y participantes impactados. No se exige un endpoint o proceso de recalcu global del torneo para cerrar el sprint.

Por que. Desde backend engineering, el recalcu selectivo reduce superficie de fallo, costo operacional y riesgo de side effects masivos. Tambien permite trazar mejor la causa entre correccion de resultado y cambio competitivo.

Impacto. Un cambio efectivo de resultado oficial debe disparar recalculacion idempotente sobre el subconjunto afectado. Si luego el producto necesita un recalc global administrativo, esa capacidad debe abrirse explicitamente en un sprint posterior.

Documento dueno. 03_Backend/tex/03_02_Procesos_de_Negocio.tex

Decision. Trigger de recalc selectivo en QH-150

Verdicto. En QH-150, el recalc selectivo se dispara tanto desde POST /matches/:matchId/result/manual como desde POST /matches/:matchId/result/sync. En ambos flujos, la condicion es estricta: solo corre cuando hubo cambio efectivo del resultado oficial vigente.

Por que. El proceso de resultados ya habia quedado cerrado como flujo manual o sincronizado, y ambos habilitan recalc cuando existe cambio efectivo. Mantener el mismo criterio evita que backend produzca comportamientos competitivos distintos segun el origen del resultado.

Impacto. Ninguno de los dos flujos puede disparar recalc por simple upsert tecnico, reescritura equivalente o metadata sin impacto competitivo.

Documento **dueno.** 03_Backend/tex/03_02_Procesos_de_Negocio.tex y
01_Producto_y_Reglas/tex/01_05_Casos_Borde.tex

Decision. Definicion de cambio efectivo en QH-150

Verdicto. Para QH-150, existe cambio efectivo unicamente cuando cambia el marcador oficial vigente del partido, es decir, `homeGoals` y/o `awayGoals`. Cambios solo en metadata interna como `source`, `recordedByUserId`, `externalReference`, `updatedAt` u otros campos auxiliares no disparan recalc.

Por que. El recalc competitivo existe para recomputar scoring cuando cambia el dato que define puntos. Permitir que metadata operativa dispare recalc introduciria ruido tecnico sin cambio real de verdad deportiva aplicada.

Impacto. Backend debe comparar el marcador oficial vigente anterior y el nuevo antes de habilitar recalc. Un rewrite del mismo score no cuenta como cambio efectivo.

Documento **dueno.** 03_Backend/tex/03_02_Procesos_de_Negocio.tex y
12_Decisiones_por_Sprint/tex/12_06_Sprint_06_contenido.tex

Decision. Alcance del recalc selectivo en QH-150

Verdicto. El recalc selectivo de QH-150 afecta exclusivamente los `ScoreEntry` de picks por partido asociados al mismo `matchId`. Debe excluir cualquier recomputacion de otros `matchId`, incluso del mismo usuario. QH-150 no implementa recalc global, por torneo ni por usuario.

Por que. Sprint 6 ya habia cerrado recalc selectivo por partido afectado y participantes impactados. Expandirlo a torneo completo, a usuario completo o a otros partidos reabriria alcance y riesgo operativo fuera del ticket.

Impacto. La unidad de recalc debe localizar solo picks y score entries del partido afectado. Cualquier necesidad de torneo completo queda fuera del ticket y debe abrirse aparte.

Documento **dueno.** 03_Backend/tex/03_02_Procesos_de_Negocio.tex y
12_Decisiones_por_Sprint/tex/12_06_Sprint_06_contenido.tex

Decision. Contrato interno minimo del recalc selectivo

Verdicto. La unidad interna reutilizable de QH-150 usa el siguiente contrato minimo obligatorio.

Input canonico.

```
{
  matchId: string;
  triggerSource: "manual" | "sync";
  cause: "official_result_changed";
  actorUserId?: string | null;
  requestId?: string | null;
}
```

Output canonico.

```
{
  matchId: string;
  affectedUsersCount: number;
  createdCount: number;
  updatedCount: number;
  unchangedCount: number;
  startedAt: Date;
  finishedAt: Date;
}
```

Por que. QH-150 necesita un contrato suficientemente explicito para ser reutilizable desde `results`, expresar idempotencia y dejar trazabilidad, sin acoplarse todavia a leaderboard final ni a contratos HTTP publicos.

Impacto. No se requieren campos adicionales obligatorios para cerrar el ticket. Si luego aparece una necesidad de observabilidad mas rica, se puede ampliar sin romper este minimo.

Documento dueno. 03_Backend/tex/03_02_Procesos_de_Negocio.tex

Decision. Trazabilidad minima de recalc selectivo

Verdicto. QH-150 debe registrar trazabilidad minima en `AuditEvent`. La accion canonica es `match_score_recalculated`. El evento debe incluir como minimo: actor identificable, `entityType = Match`, `entityId = matchId`, `requestId` cuando exista, y un `metadataJson` con `triggerSource`, `cause`, `affectedUsersCount`, `createdCount`, `updatedCount`, `unchangedCount`, `startedAt` y `finishedAt`.

Por que. El ticket exige trazabilidad minima del recalc. A la vez, el modelo de auditoria del proyecto ya concentra evidencia de procesos criticos sin convertir cada entidad derivada en una historia versionada propia.

Impacto. QH-150 no necesita persistir before/after por usuario dentro del evento minimo. La auditoria del intento y sus contadores es suficiente para el cierre del ticket.

Documento dueno. 05_Datos/tex/05_04_Auditoria_y_Trazabilidad.tex y 03_Backend/tex/03_02_Procesos_de_Negocio.tex

Decision. Politica transaccional del recalc selectivo

Verdicto. En QH-150, el resultado oficial vigente se persiste primero. El recalc selectivo corre inmediatamente despues como paso separado del flujo de aplicacion. Si el recalc falla, el resultado oficial no se revierte: la verdad oficial vigente se conserva y el sistema reporta error operativo con trazabilidad.

Por que. El modulo **results** ya es autoridad de la verdad oficial vigente. Revertir el resultado por un fallo posterior de recalc haria que un problema operativo de scoring borre una verdad oficial ya consolidada, lo cual contradice la separacion de responsabilidades del dominio.

Impacto. El sistema puede quedar temporalmente con resultado oficial persistido y recalc pendiente o fallido, pero nunca debe sacrificar la verdad oficial vigente por una falla de recomputacion derivada.

Documento dueno. 03_Backend/tex/03_02_Procesos_de_Negocio.tex y 06_Integraciones/tex/06_02_Sincronizacion_y_Fallback_Manual.tex

Decision. Error publico ante fallo de recalc selectivo

Verdicto. Si QH-150 falla al ejecutar recalc selectivo, el codigo publico aplicable es **internal_error**. La respuesta publica no expone detalle fino adicional; ese detalle vive en logging y auditoria interna.

Por que. Un fallo de recalc posterior a persistencia del resultado oficial es un problema operativo interno, no una nueva regla funcional para el usuario. Exponer detalles de implementacion no aporta valor funcional y aumenta ruido contractual.

Impacto. Backend conserva el envelope estandar del proyecto y deriva el diagnostico fino a observabilidad interna.

Documento dueno. 05_Datos/tex/05_04_Auditoria_y_Trazabilidad.tex y envelope estandar vigente del backend

Decision. Idempotencia operativa del recalc selectivo

Verdicto. Si QH-150 se ejecuta dos veces sobre el mismo estado final de resultado, la segunda ejecucion debe converger sin cambios efectivos de **ScoreEntry**. En ese caso, el output esperado es **createdCount = 0**, **updatedCount = 0** y **unchangedCount = affectedUsersCount**. El intento igualmente deja trazabilidad.

Por que. El dominio ya habia cerrado que multiples recalculos sobre el mismo partido corregido deben converger al mismo estado final. Expresar esta convergencia con contadores explicitos evita ambiguedad tecnica y facilita pruebas.

Impacto. Un re-run idempotente no se trata como error ni como no-op invisible; se trata como ejecucion valida sin cambios efectivos.

Documento dueno. 03_Backend/tex/03_02_Procesos_de_Negocio.tex y 01_Producto_y_Reglas/tex/01_05_Casos_Borde.tex

Decision. Relacion de QH-150 con predicciones especiales

Verdicto. QH-150 recalcula solo `ScoreEntry` de picks por partido. Debe excluir cualquier `specialPredictionId`. No hay excepciones en este ticket.

Por que. El trigger, el alcance selectivo y el ticket mismo estan definidos por cambio de resultado oficial de un partido. Las predicciones especiales no dependen del `matchId` corregido bajo este alcance.

Impacto. La logica de QH-150 opera sobre score entries vinculados al partido afectado y deja fuera scoring especial hasta el ticket que corresponda.

Documento dueno. 10_Planeacion/Sprint_06_Jira_Ticket_Drafts.md y 01_Producto_y_Reglas/tex/01_04_Scoring_y_Desempates.tex

Decision. Evidencia minima backend para QH-150

Verdicto. Para considerar QH-150 cerrado en backend, la evidencia minima obligatoria debe cubrir:

- cambio efectivo manual que recalcula solo el `matchId` afectado;
- cambio efectivo por sync que recalcula solo el `matchId` afectado;
- no-op competitivo cuando solo cambia metadata sin cambio de marcador;
- segunda ejecucion idempotente sobre el mismo estado final con cero cambios efectivos;
- aislamiento: partidos no relacionados permanecen intactos;
- exclusion de `specialPredictionId`;
- registro de `AuditEvent` con `action = match_score_recalculated`.

Por que. Es el minimo que demuestra trigger correcto, alcance selectivo, convergencia, trazabilidad y ausencia de side effects fuera del partido afectado.

Impacto. QH-150 no requiere frontend para validarse. Su barra de aceptacion backend vive en pruebas unitarias e integracion del flujo interno.

Documento dueno. 08_Testing/tex/08_03_Criterios_de_Aceptacion.tex, 03_Backend/tex/03_02_Procesos_de_Negocio.tex y 12_Decisiones_por_Sprint/tex/12_06_Sprint_06_c

1.5 Scoring persistido y edge cases

Decision. Partido cancelado con resultado persistido

Verdicto. Si un partido permanece en estado `CANCELADO`, ese partido no es utilizable para scoring aunque exista un `Result` persistido. Ese resultado puede existir como dato operativo o inconsistente, pero no habilita puntaje mientras el estado funcional siga siendo `CANCELADO`.

Por que. La documentacion oficial ya define `Cancelado` como partido sin resultado oficial utilizable para la quiniela. Ademias, el dominio fija como invariante que un partido cancelado no produce puntaje. Permitir scoring por el solo hecho de tener un `Result` persistido introduciria una segunda semantica no documentada para `CANCELADO`.

Impacto. Si aparece un `Result` persistido sobre un partido `CANCELADO`, el caso se trata como inconsistencia operativa y requiere correccion administrativa auditada antes de reabrir scoring. Persistido no equivale por si solo a utilizable.

Documento dueno. 01_Producto_y_Reglas/tex/01_05_Casos_Borde.tex, 05_Datos/tex/05_03_Invariantes_de_Datos.tex y 03_Backend/tex/03_02_Procesos_de_Negocio.tex

Decision. Autoridad operativa entre estado de partido y resultado persistido

Verdicto. Para scoring no basta con que exista un **Result** persistido. El backend solo puede puntuar cuando existe una verdad oficial vigente utilizable y esa verdad es consistente con el estado funcional del partido. Si **match.status** y **Result** se contradicen, no se puntua automaticamente; primero se corrige la inconsistencia con trazabilidad.

Por que. El proceso de scoring parte de un resultado oficial vigente utilizable, pero el modelo de estados y edge cases sigue restringiendo cuando un partido puede considerarse competitivo a efectos de puntaje. Separar persistencia de utilabilidad evita que una escritura de datos se convierta accidentalmente en autorizacion competitiva.

Impacto. El modulo **results** puede conservar y exponer la verdad oficial vigente, pero el modulo **scoring** debe rechazar o saltar casos con contradiccion documental entre estado funcional y resultado consumible. La resolucion del conflicto pertenece a operacion administrativa auditada, no al scoring automatico.

Documento **dueno.** 03_Backend/tex/03_02_Procesos_de_Negocio.tex,
01_Producto_y_Reglas/tex/01_02_Estados_y_Flujos.tex y
01_Producto_y_Reglas/tex/01_05_Casos_Borde.tex

Decision. Tipo persistente de **ScoreEntry.points**

Verdicto. En Sprint 6, **ScoreEntry.points** debe persistirse como **Float**. Mantenerlo como **Int** contradice el scoring oficial cuando el puntaje derivado tiene componente fraccionario por multiplicador de fase.

Por que. El dominio ya cerro que el calculo conserva precision exacta y no debe redondearse internamente. Ademas, la implementacion de scoring por partido ya produce valores como 6.25, 4.5 y 5.25. Si la persistencia se mantiene en **Int**, backend se veria forzado a truncar o redondear un valor que la regla oficial exige conservar.

Impacto. El ticket de persistencia de **ScoreEntry** debe incluir migracion de esquema para reemplazar **Int** por **Float** en **points**. Este cambio no abre una tabla paralela ni redefine la formula de scoring; solo alinea la capa de datos con el valor oficial ya cerrado por producto y calculado por backend.

Documento **dueno.** 01_Producto_y_Reglas/tex/01_04_Scoring_y_Desempates.tex,
10_Planeacion/Sprint_06_Jira_Ticket_Drafts.md y runtime de QH-147

Decision. Contrato de **ScoreEntry.reason**

Verdicto. En Sprint 6, **ScoreEntry.reason** es obligatorio, no puede quedar vacio y no se cierra como enum oficial en esta fase. Su contrato minimo es conservar contexto funcional suficiente para explicar el origen del puntaje derivado.

Por que. La documentacion del ticket exige **reason** y trazabilidad funcional, pero no fija un catalogo cerrado. Forzar un enum nuevo sin fuente duena reabriria una decision que los documentos actuales no cerraron explicitamente.

Impacto. Backend debe persistir **reason** en create y update de **ScoreEntry**. Los tickets de Sprint 6 no deben asumir que existe ya un enum oficial ni abrir un contrato mas rigido sin documentarlo en una fuente de dominio o datos.

Documento **dueno.** 10_Planeacion/Sprint_06_Jira_Ticket_Drafts.md y
05_Datos/tex/05_02_Entidades_y_Relaciones.tex

Decision. Contenido minimo de `ScoreEntry.reason`

Verdicto. Aunque `reason` no se cierre como enum oficial en Sprint 6, el valor persistido debe dejar claro, como minimo, cual fue la salida funcional del scoring por partido. Debe distinguir al menos: ausencia valida de pick, outcome correcto, marcador exacto y partido cancelado sin resultado utilizable.

Por que. Esas distinciones ya aparecen como salidas obligatorias del calculo y como casos minimos de cobertura. Exigirlas dentro de `reason` alinea persistencia, trazabilidad funcional y testing sin imponer todavia una taxonomia de dominio mas amplia que la documentada.

Impacto. Las implementaciones de Sprint 6 pueden resolver `reason` como texto controlado o formato interno equivalente, siempre que cubran como minimo esas categorias funcionales y no dejen razones opacas o vacias.

Documento **dueno.** 10_Planeacion/Sprint_06_Jira_Ticket_Drafts.md y
03_Backend/tex/03_02_Procesos_de_Negocio.tex

Decision. Politica de actualizacion de `ScoreEntry.reason`

Verdicto. `ScoreEntry` mantiene una sola fila vigente por combinacion competitiva documentada en el ticket. Si cambia el puntaje derivado, la misma fila se actualiza y `reason` pasa a reflejar el motivo vigente final. El historial de cambios no se versiona dentro de `ScoreEntry`; vive en auditoria.

Por que. Sprint 6 ya habia cerrado unicidad por entrada y comportamiento `update`, no `create`, cuando cambia el valor derivado. A la vez, la trazabilidad historica del sistema pertenece al subdominio de auditoria. Duplicar o versionar razones dentro de la misma entidad mezclaria scoring derivado con historia operativa.

Impacto. Recalculo y correccion de resultado deben converger sobre una unica fila vigente de `ScoreEntry`. Si se necesita reconstruir el antes y despues, la fuente es `AuditEvent` y no un versionado embebido del score derivado.

Documento **dueno.** 10_Planeacion/Sprint_06_Jira_Ticket_Drafts.md,
05_Datos/tex/05_04_Auditoria_y_Trazabilidad.tex y 03_Backend/tex/03_02_Procesos_de_Negocio.tex

1.6 Leaderboard backend y desempates cerrados

Decision. Contrato backend oficial de leaderboard general en QH-151

Verdicto. El contrato oficial de Sprint 6 para leaderboard general es GET /leaderboard autenticado, con respuesta oficial ordenada desde backend y sin recalcule local en cliente. Cada fila expone como minimo `position`, `participant.userId`, `participant.displayName` y `score.total`. El ranking incluye solo usuarios ACTIVO, incluyendo activos con 0 puntos.

Por que. La documentacion de dominio y backend ya cierra que el ranking oficial no puede depender del cliente y que la vista general usa acumulado oficial del torneo. Para evitar ambigüedad de implementacion, el ticket debe fijar ruta, shape y semantica de inclusion sin dejar decisiones grandes al implementador.

Impacto. Backend implementa contrato estable consumible por frontend placeholder sin reinterpretar scoring. QH-151 no introduce filtros, paginacion ni breakdown obligatorio por fila; solo cierra lectura general oficial reproducible.

Documento dueno. 01_Producto_y_Reglas/tex/01_04_Scoring_y_Desempates.tex, 03_Backend/tex/03_04_Endpoints_Conceptuales.tex, 05_Datos/tex/05_03_Invariantes_de_Datos.tex y 12_Decisiones_por_Sprint/tex/12_06_Sprint_06_contenido.tex

Decision. Orden operativo base en QH-151 hasta aplicar QH-152

Verdicto. Mientras QH-152 no este aplicado, QH-151 usa orden base determinista para empates de total: `score.total` DESC, luego `user.createdAt` ASC y finalmente `user.id` ASC. La `position` es ordinal unica secuencial 1..n consistente con el orden final.

Por que. QH-151 debe poder cerrar contrato sin dejar orden arbitrario, pero no reabre la jerarquia competitiva oficial que pertenece a QH-152. Este orden base evita nondeterminismo y da estabilidad contractual mientras entra el algoritmo completo de desempates.

Impacto. El backend de QH-151 entrega ranking reproducible desde dia uno y QH-152 pasa a ser un reemplazo controlado del comparador, no una redefinicion del contrato HTTP.

Documento dueno. 05_Datos/tex/05_03_Invariantes_de_Datos.tex, 03_Backend/tex/03_02_Procesos_de_Negocio.tex y tickets QH-151/QH-152

Decision. Jerarquia oficial de desempates en QH-152

Verdicto. El orden oficial en empate de total se aplica exactamente asi: mayor `EXACT_SCORE`, luego mayor cantidad de outcomes acertados, luego mayor puntaje KO, luego registro mas temprano y, si persiste empate perfecto, fallback tecnico final por `user.id` ASC. Este fallback no altera la jerarquia competitiva; solo garantiza orden total determinista.

Por que. El documento dueno ya define la jerarquia competitiva oficial. El ticket debe aterizarla a un comparador completamente ejecutable para eliminar interpretaciones diferentes entre implementaciones y ambientes.

Impacto. QH-152 no cambia ruta ni shape de respuesta de QH-151; cambia solo el comparador interno del orden. El ranking final queda reproducible y explicable a partir de datos fuente vigentes.

Documento dueno. 01_Producto_y_Reglas/tex/01_04_Scoring_y_Desempates.tex, 03_Backend/tex/03_02_Procesos_de_Negocio.tex y 05_Datos/tex/05_03_Invariantes_de_Datos.tex

Decision. Semantica operativa de conteos de desempate en QH-152

Verdicto. Para QH-152, `EXACT_SCORE` cuenta tambien como acierto de outcome. Los conteos de desempate se calculan sobre `ScoreEntry` de partido (`matchId != null`). Las predicciones especiales suman al total general, pero no participan en criterios de desempate. KO se interpreta como fases `Octavos de final`, `Cuartos de final`, `Semifinales`, `Tercer lugar` y `Final`. La fecha de registro para desempate es `User.createdAt`.

Por que. Sin esta capa operativa, la jerarquia oficial seguiria teniendo huecos de implementacion: que cuenta como outcome, que entra en KO y que fuente de fecha se usa. Cerrar estos puntos evita drift funcional entre repositorios y pruebas.

Impacto. Backend puede implementar y probar QH-152 sin abrir decisiones adicionales. La evidencia minima de testing pasa a ser verificable con datos concretos y resultados esperados univocos.

Documento dueno. `01_Producto_y_Reglas/tex/01_04_Scoring_y_Desempates.tex`, `10_Planeacion/Sprint_06_Jira_Ticket_Drafts.md` y contratos de `scoring` en backend

Decision. Modelo semantico de fase y jornada para QH-153

Verdicto. Para Sprint 6, `fase` y `jornada` son conceptos distintos. `fase` se implementa con la entidad `Phase`. `jornada` no se deriva de `Phase.order`, `Phase.name` ni de heurísticas por fecha; se implementa como dato persistido `Match.matchday`. El concepto de producto sigue llamandose `jornada`; el nombre tecnico del campo y del filtro de API es `matchday`.

Por que. El dominio y Sprint 6 ya esperan dos cortes distintos del leaderboard: por fase y por jornada. Si ambos conceptos se colapsan sobre `Phase`, el sistema finge dos filtros cuando en realidad tendria uno solo. Persistir `matchday` en `Match` mantiene separacion semantica clara, evita heurísticas inventadas y sigue principios de simplicidad operativa sin sobrediseñar una entidad nueva.

Impacto. Backend puede filtrar por fase usando `phaseId` y por jornada usando `matchday`. El proveedor externo puede seguir entregando `matchday` como dato de integracion, pero la fuente oficial del filtro pasa a ser el campo persistido del dominio. Frontend y producto muestran `jornada` al usuario; API, OpenAPI y tests usan `matchday`.

Documento dueno. `03_Backend/tex/03_04_Endpoints_Conceptuales.tex`, `06_Integraciones/tex/06_01_API_de_Resultados.tex`, `QuinielaBackend/prisma/schema.prisma` y `QuinielaBackend/src/modules/matches`

Decision. Contrato HTTP exacto de leaderboard filtrado en QH-153

Verdicto. QH-153 reutiliza GET /leaderboard y no crea rutas nuevas. El contrato acepta query params mutuamente excluyentes: `phaseId` como `string` no vacío o `matchday` como entero positivo. Sin filtros, la respuesta sigue siendo el leaderboard general. Si se envían ambos filtros a la vez, el backend responde 400 `validation_error`. La autenticación y autorización son exactamente las mismas que en QH-151: cualquier usuario autenticado puede consultar el leaderboard general o filtrado.

Por que. El sistema ya tiene una sola superficie conceptual de leaderboard y un endpoint general existente. Mantener la misma ruta con query params evita drift entre frontend, backend y OpenAPI, sigue la convención ya usada por `matches` y reduce complejidad accidental del contrato.

Impacto. Frontend cambia de modo general, por fase o por jornada sin navegar a endpoints distintos. OpenAPI y Postman deben documentar: request sin filtros, request con `phaseId`, request con `matchday` y error cuando ambos llegan juntos.

Documento dueño. 03_Backend/tex/03_04_Endpoints_Conceptuales.tex, QuinielaBackend/src/modules/leaderboard/http/routes.ts, QuinielaBackend/src/modules/matches/http/schemas.ts y QuinielaBackend/docs/api/openapi.yaml

Decision. Semántica exacta de filtros y shape de fila en QH-153

Verdicto. En filtro por fase, el leaderboard suma solo `ScoreEntry` de partido cuyo `Match.phaseId` coincide con el `phaseId` consultado. En filtro por jornada, suma solo `ScoreEntry` de partido cuyo `Match.matchday` coincide con el `matchday` consultado. Un partido sin `matchday` no participa en un leaderboard por jornada. La fila filtrada mantiene exactamente el mismo shape base del leaderboard general: `position`, `participant.userId`, `participant.displayName` y `score.total`. En una respuesta filtrada, `score.total` significa el puntaje del scope consultado y no el total general del torneo.

Por que. Sprint 6 ya cerro que el valor principal visible de una vista filtrada debe corresponder solo al scope activo. Mantener el mismo shape evita bifurcar contratos y obliga a que la diferencia de semántica viva en el scope de la consulta, no en un cambio arbitrario de estructura.

Impacto. El cliente no necesita adaptadores especiales por tipo de leaderboard. Sin embargo, backend y docs deben dejar explícito que `score.total` cambia de significado entre vista general y vista filtrada según el scope consultado.

Documento dueño. 01_Producto_y_Reglas/tex/01_04_Scoring_y_Desempates.tex, 03_Backend/tex/03_04_Endpoints_Conceptuales.tex, QuinielaBackend/src/modules/leaderboard/http y ticket QH-153

Decision. Predicciones especiales, desempates y usuarios con cero en QH-153

Verdicto. En leaderboard filtrado, las predicciones especiales quedan completamente fuera del response: no entran al puntaje principal, no entran a desempates y no se muestran como breakdown o contexto secundario. Los desempates reutilizan la jerarquia de QH-152, pero calculada solo con `ScoreEntry` dentro del scope filtrado. No se usa total general como fallback intermedio. Los usuarios `ACTIVO` siguen apareciendo aunque tengan 0 puntos en el scope filtrado.

Por que. Las predicciones especiales pertenecen al acumulado del torneo completo y mezclar su presencia dentro de una vista filtrada reintroduce ruido semantico. A la vez, si el filtro representa un ranking propio, sus desempates deben salir de ese mismo subconjunto y no de informacion externa al corte consultado. Mantener usuarios activos con 0 puntos conserva un universo competitivo estable entre vistas.

Impacto. Backend calcula orden oficial del filtro sobre entries del scope y no sobre el total general. Frontend puede alternar entre general y filtros sin que desaparezcan participantes por una regla distinta de inclusion.

Documento dueno. 01_Producto_y_Reglas/tex/01_04_Scoring_y_Desempates.tex, 12_Decisiones_por_Sprint/tex/12_06_Sprint_06_contenido.tex y contratos cerrados en QH-151/QH-152

Decision. Errores, seed minima y alcance tecnico de QH-153

Verdicto. En QH-153, un `phaseId` con formato invalido responde 400 `validation_error`; un `phaseId` bien formado pero inexistente responde 404 `not_found`. Un `matchday` con formato invalido responde 400 `validation_error`; un `matchday` valido sin partidos o sin score asociado responde 200 OK con `leaderboard = []`. El ticket incluye backend query/service/repository, tests unitarios e integracion, actualizacion de OpenAPI, actualizacion de Postman y seed minima reproducible para al menos dos fases y dos jornadas.

Por que. `phaseId` referencia una entidad explicita del dominio y por eso su inexistencia debe ser tratada como recurso no encontrado. `matchday`, en cambio, funciona como valor de corte y no como recurso direccionable propio, asi que la ausencia de datos se expresa mejor con lista vacia. Como el ticket expone contrato backend consumible, query sin documentacion, ejemplos y dataset reproducible dejaria el cierre incompleto.

Impacto. Implementacion y QA cuentan con reglas estables para validar errores, fixtures y consumo manual. QH-153 no se limita a query mas tests: sale listo para integracion con frontend, OpenAPI y Postman sin interpretaciones adicionales.

Documento dueno. ticket QH-153, QuinielaBackend/docs/api/openapi.yaml, QuinielaBackend/prisma/seed.ts y politica de errores del backend

1.7 Fronteras tecnicas obligatorias

Decision. Autoridad oficial de scoring y ranking

Verdicto. El backend es la unica autoridad oficial para score visible, desempates y orden del leaderboard. El frontend no puede calcular ranking oficial ni reordenar participantes usando logica propia.

Por que. La coherencia competitiva exige una sola fuente de verdad. Permitir calculo paralelo en cliente abriria drift entre usuarios, bugs de consistencia y tickets ambiguos. Los helpers locales solo son aceptables como apoyo de presentacion no oficial y nunca como fuente de ranking.

Impacto. Los tickets de Sprint 6 deben tratar cualquier helper local de scoring como no autoritativo. La UI consume el derivado oficial del backend para total, posicion y filtros.

Documento dueno. 03_Backend/tex/03_02_Procesos_de_Negocio.tex y 04_Frontend/tex/04_03_Estado_Validacion_y_Datos.tex

Decision. Politica de refresco frontend tras correccion de resultado

Verdicto. Ante una correccion manual o una sincronizacion externa con cambio efectivo, el frontend debe invalidar al menos:

- resultado visible del partido afectado;
- calendario o listado que contiene ese partido;
- score visible del usuario si existe en la UI;
- leaderboard activo y sus filtros visibles;
- detalle de partido o detalle de rendimiento dependiente del cambio.

Por que. Desde software engineering y UX, mostrar score o ranking obsoleto despues de una correccion rompe confianza y genera inconsistencia observable. La politica de invalidacion debe ser explicita para que el cliente no dependa de recargas manuales ni de campos no garantizados.

Impacto. Frontend no debe asumir banderas como `hasImpact` o `isCorrected` si backend no las expone formalmente. La reconciliacion se hace por refresco de datos oficiales y no por inferencia local.

Documento dueno. 04_Frontend/tex/04_03_Estado_Validacion_y_Datos.tex

1.8 Alineaciones obligatorias antes de ticketear Sprint 6

- Los tickets de Sprint 6 no deben asumir campos o rutas de `results` que el backend no haya formalizado en contrato.
- Los tickets de Sprint 6 no deben reabrir formula de scoring, multiplicadores ni desempates ya cerrados en `01_04_Scoring_y_Desempates.tex`.
- Si existe drift documental heredado de Sprint 5 sobre `results`, el trabajo de Sprint 6 debe escalarlo y sincronizarlo; no puede improvisar una tercera variante.

1.9 Notas

- Estas decisiones cierran los pendientes necesarios para redactar tickets de Sprint 6 sin dejar huecos de UX, recalcular o autoridad de scoring.

- La implementacion de Sprint 6 debe seguir principios de HCI: una sola superficie clara por objetivo, feedback visible, baja carga cognitiva y consistencia entre score mostrado y ranking recibido.
- La implementacion de Sprint 6 debe seguir principios de software engineering: autoridad unica del backend, contratos explicitos, recalcule idempotente, filtros no ambiguos y ausencia de logica competitiva oficial en cliente.